

Secure On-Premise SLM-Based Chatbot for Sales, Inventory & Financial Analysis using Gemma 4B

Executive Summary:

This project presents the design and implementation of a fully on-premise AI chatbot system built using the Gemma 4B Small Language Model (SLM) running on Ollama, with a Python Flask-based application layer.

The chatbot enables users to analyze Sales, Inventory, and Financial data stored in MySQL and other RDBMS systems using natural language queries, while strictly enforcing Role-Based Access Control (RBAC) and Row-Level Security (RLS).

The system is designed for high-security environments where no cloud usage, no internet connectivity, and no external data sharing are allowed.

All AI inference, data processing, learning, and storage occur entirely within the organization's internal infrastructure, and the complete source code is handed over after deployment.

Scope of the System:

- On-premise Gemma 4B model using Ollama
- Flask-based chatbot web application
- Sales, Inventory, and Finance analytics
- Secure Text-to-SQL generation
- Role-based and row-level data security
- Fine-tuning with custom datasets
- Controlled learning from SQL data
- Logging, auditing, and monitoring
- Full source code and documentation

Core Technology Stack:

AI & Model Layer

- Model: **Gemma 4B**
- Runtime: **Ollama**
- Inference: Offline, on-premise
- Learning: LoRA-based fine-tuning

Backend / Application

- Language: Python
- Framework: Flask
- Authentication: JWT / LDAP
- Business logic & validation layer

Database Layer

- MS SQL
- Views, stored procedures, RLS logic

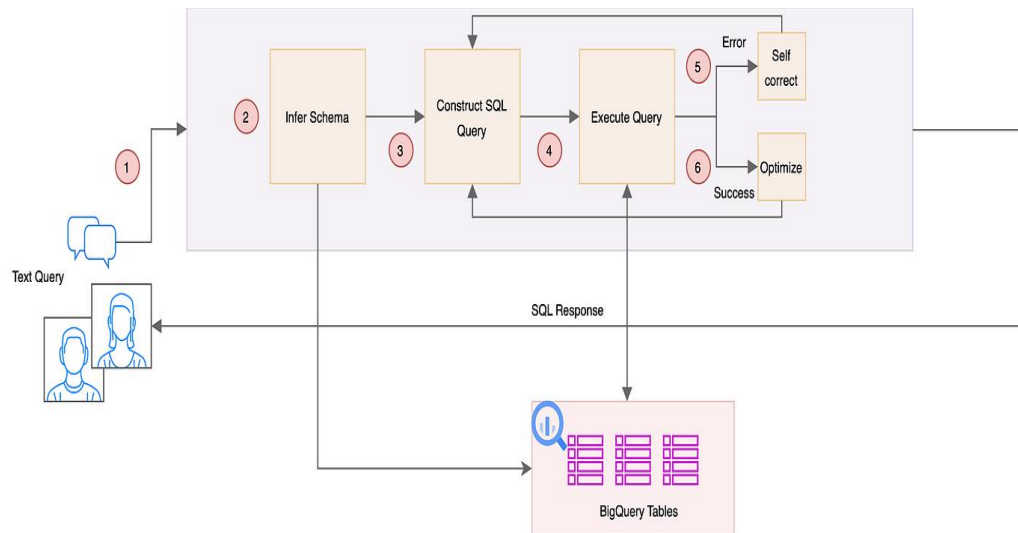
Frontend

- Flask-rendered UI / HTML, CSS, JavaScript
- Chat-style interface
- Role-aware responses

Real-Time Learning (RAG / Prompt Injection)

- Database schemas
- KPI definitions
- Business rules
- Injected dynamically into prompts

High-Level Architecture:



Security Design:

Authentication

- Internal user authentication
- Secure token-based sessions (JWT)

Authorization (RBAC)

- Roles defined per department
- Role validated on every request

Row-Level Security (RLS)

- Implemented at database level
- Users can access only authorized rows
- Column masking for sensitive finance data

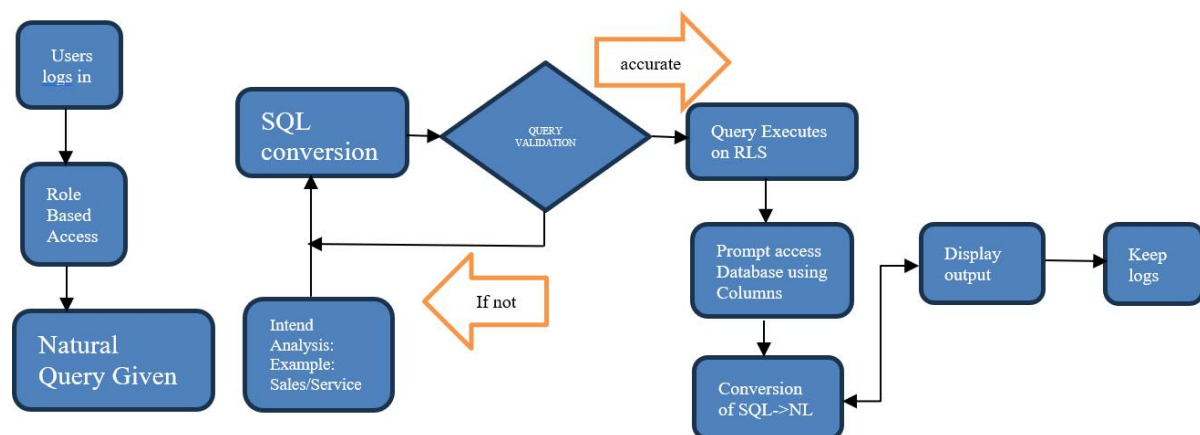
Secure Coding Practices

- Parameterized SQL queries
- ORM-based access
- SQL injection prevention
- Prompt injection protection
- Read-only database credentials for AI

Role-Based Access Model:

Role	Access Scope
Admin	System monitoring
Sales User	Sales data (assigned region only)
Inventory User	Inventory & stock data
Finance User	Financial & accounting data
Management	Aggregated insights

Chatbot Working Mechanism:



Model Fine-Tuning Strategy:

Purpose of Fine-Tuning

- Improve understanding of business terminology
- Improve Text-to-SQL accuracy
- Align model behaviour with enterprise rules

Dataset Format:

Instruction-based JSONL:

```
{
  "instruction": "Show last month sales for South region",
  "input": "",
  "output": "SELECT SUM(amount) FROM sales WHERE region='South';"
}
```

Audit & Monitoring:

- User activity logs
- SQL query logs
- Model response logs
- Access violation alerts

Hardware Requirements:

Based on number of users accessing

Deliverables:

- Deployed on-premise chatbot
- Gemma 4B custom model
- Backend & frontend source code
- Deployment documentation
- Knowledge transfer session

This project delivers a secure, enterprise-grade, on-premise AI chatbot built using Gemma 4B on Ollama and Flask. By combining controlled fine-tuning, secure SQL access, RBAC, and RLS, the solution enables powerful conversational analytics without compromising data security or compliance.